Objetivos:

I. Introdução ao Redux;
II. Conceitos fundamentais do Redux;
III. Vite;

I. Introdução ao Redux

Em aplicações React, o estado é uma peça fundamental para a construção de interfaces dinâmicas. O Redux é uma biblioteca projetada para gerenciar estados globais de forma centralizada e previsível. Ele é baseado em três pilares principais:

1. Store (armazenamento): um único objeto JS que contém o estado global da aplicação;
2. Actions (ações): objetos que descrevem "o que aconteceu" na aplicação;
3. Reducers (redutores): funções puras que recebem o estado atual e uma ação, retornando um novo estado.

Esses princípios fazem do Redux uma ferramenta poderosa para:

- Organizar o estado de forma clara e centralizada;
- Escalar aplicações com muitos estados e interações;
- Depurar facilmente, graças a ferramentas como Redux DevTools.

Comparação entre Redux e Context API

O Context e o Redux são ambos mecanismos para gerenciar estado em aplicações React, mas possuem abordagens e aplicações diferentes:

Característica	**Redux**	**Context**
Propósito	É ideal para gerenciar estados complexos e dinâmicos, como listas de produtos, carrinhos de compras ou estados sincronizados com back-end.	Foi projetado para compartilhar dados simples (como tema, idioma ou informações do usuário logado). É uma ferramenta leve, sem sobrecarga significativa.
Estrutura	Store, actions, reducers	Provider e Consumer
Complexidade do estado	Oferece uma estrutura mais robusta, permitindo melhor organização de estados complexos por meio de reducers e middlewares.	Torna-se difícil de manter em casos de estados grandes ou muitos "dispatches" diferentes.
Escalabilidade	Permite otimizações granulares, garantindo que apenas os componentes afetados sejam re-renderizados.	Funciona bem em aplicações pequenas e médias, mas pode sofrer problemas de performance em atualizações frequentes, já que a re-renderização pode afetar vários componentes.
Desempenho	Pode ter um impacto maior no desempenho em aplicações muito grandes	Geralmente tem um desempenho melhor
Aprendizado	Requer mais aprendizado	Mais fácil de aprender

II. Conceitos fundamentais do Redux

Para compreender como o Redux funciona e como ele gerencia o estado global de forma centralizada, é necessário entender os principais elementos que compõem sua arquitetura: Store, Actions, Reducers e Dispatch. Esses conceitos trabalham juntos para criar um fluxo previsível e organizado de dados na aplicação.

Store: O armazenamento central

A Store é o coração do Redux. Ela é responsável por armazenar o estado global da aplicação e agir como a única fonte de verdade. Isso significa que qualquer dado compartilhado entre componentes será mantido na Store.

Características da Store:

- Contém o estado global da aplicação;
- Permite o acesso ao estado por meio do método getState();
- Disponibiliza o método dispatch(action) para enviar ações e modificar o estado;
- Oferece o método subscribe(listener) para reagir a mudanças no estado.

Exemplo de criação de uma Store:

```
import { createStore } from 'redux';
import { rootReducer } from './reducers';

const store = createStore(rootReducer);
```

Actions: Descrevendo o que deve acontecer

As Actions são objetos simples que descrevem uma intenção de modificar o estado. Elas contêm uma propriedade obrigatória chamada type, que indica o tipo de ação a ser executada, e podem conter outros dados relacionados à ação.

Estrutura de uma Action:

```
type Action = {
  type: string;
  payload?: User; // Dados adicionais que podem ser enviados junto com a ação
};
```

Exemplo de uma Action:

```
const userAction = {
  type: "string",
  payload: {
    id: 1,
    name: "Ana",
    age: 21
  },
};
```

As Actions são apenas descrições do que deve acontecer. Elas não alteram o estado diretamente.

Reducers: Funções puras que atualizam o estado

Os Reducers são funções puras responsáveis por processar uma Action e retornar um novo estado. Eles recebem o estado atual (`state`) e a `action` como parâmetros e, com base no tipo da ação, retornam o estado atualizado.

Características dos Reducers:

- São funções puras, ou seja, não produzem efeitos colaterais;
- Não devem modificar diretamente o estado; ao invés disso, devem retornar um novo objeto com as alterações.

Exemplo de um Reducer:

```
const userSlice = createSlice({
  name: "cadastro",
  initialState: {
    users: [],
  } as UserState,
  reducers: {
    removeUser: (state, action: PayloadAction<number>) => {
      // Filtra o array e atualiza o estado
      state.users = state.users.filter((user) => action.payload !== user.id);
    },
  },
});
```

Dispatch: Disparando ações

O Dispatch é o método utilizado para enviar Actions para a Store. Quando uma Action é disparada, a Store repassa essa Action para o Reducer, que calcula o novo estado.

```
const dispatch: AppDispatch = useDispatch();
dispatch(removeUserAndCars(id));
```

Como o Redux gerencia o fluxo de dados

O Redux segue um fluxo de dados unidirecional bem definido, o que facilita a previsibilidade e o rastreamento do estado. Esse fluxo pode ser resumido em quatro etapas:

1. Disparar uma ação: um componente dispara uma action usando o método `dispatch()`;

```
const dispatch: AppDispatch = useDispatch();
dispatch(removeUserAndCars(id));
```

2. Processar no Reducer: a Store invoca o `reducer`, que calcula o novo estado com base na `action`;

```
removeUser: (state, action: PayloadAction<number>) => {
  state.users = state.users.filter((user) => action.payload !== user.id);
},
```

3. Atualizar a Store: o novo estado retornado pelo Reducer substitui o estado anterior na Store;

4. Notificar os componentes: todos os componentes que dependem do estado atualizado são notificados e re-renderizados, se necessário.

Diagrama do fluxo de dados:

Component -> dispatch(Action) -> Reducer -> Store -> Component

III. Vite

O Vite é uma ferramenta de construção de aplicações web de nova geração que se destaca pela sua velocidade e leveza. Ele oferece um servidor de desenvolvimento instantâneo e hot reloading extremamente rápido, o que torna o processo de desenvolvimento muito mais ágil e prazeroso.

- **Velocidade:** O Vite é significativamente mais rápido do que outras ferramentas de construção, como o Webpack. Ele usa módulos ES nativos para servir o código durante o desenvolvimento, o que elimina a necessidade de empacotamento.
- **Hot Reloading:** O hot reloading do Vite é extremamente rápido. As alterações no código são refletidas instantaneamente no navegador, sem a necessidade de recarregar a página.
- **Facilidade de uso:** O Vite é muito fácil de configurar e usar. Ele oferece uma experiência de desenvolvimento intuitiva e agradável.
- **Suporte a TypeScript:** O Vite oferece suporte nativo ao TypeScript, o que facilita o desenvolvimento de aplicações React com tipagem estática.